

2025년 1회 정보처리기사 실기 복원 문제 (해설 추가본)

출처: <https://chobopark.tistory.com/540> (Life-Journey)

1번

다음은 네트워크 보안에 관련된 문제이다. 괄호 안에 알맞는 용어를 작성하시오.

()은/는 '세션을 가로채다.'라는 의미로 다른 사람의 세션 상태를 훔치거나 도용하여 액세스하는 해킹 기법이다. TCP ()은/는 3-way 핸드셰이크가 완료된 후 공격자가 시퀀스 번호 등을 조작하여 정상 세션을 가로채고 인증 없이 통신을 탈취하는 공격이다.

정답: 세션 하이재킹

해설:

세션 하이재킹(Session Hijacking)은 이미 인증을 마친 사용자의 **세션(연결 상태)**을 가로채서 그 사람인 척 통신을 탈취하는 공격입니다.

- '세션'이란 로그인 후 서버가 "이 사용자는 인증됨"이라고 기억하는 연결 상태입니다.
- TCP는 연결할 때 **3-way 핸드셰이크**(SYN → SYN/ACK → ACK)로 시작하고, 각 패킷에 **시퀀스 번호**를 붙여 순서를 관리합니다.
- 공격자는 이 시퀀스 번호를 알아내 조작함으로써, 정상 사용자가 인증한 세션에 끼어들어 **별도 인증 없이** 통신을 빼앗습니다.

키워드 "세션 탈취 + 시퀀스 번호 조작 = 세션 하이재킹"으로 기억하세요.

2번

다음은 제약조건과 관련된 문제이다. 괄호 안에 알맞는 용어를 보기에서 골라 작성하시오. (※ 원문에 이미지 포함)

[보기] 개체, 참조, 도메인

정답: ㄱ. 도메인 / ㄴ. 개체 / ㄷ. 참조

해설:

데이터베이스의 **무결성 제약조건 3종**을 구분하는 문제입니다.

- 도메인 무결성(Domain Integrity)**: 각 속성(컬럼) 값은 **정해진 자료형·범위·형식**에 맞아야 합니다. 예: 나이 컬럼에 문자열이나 음수가 들어가면 위반.
- 개체 무결성(Entity Integrity)**: 기본 키는 **NULL이 될 수 없고 중복될 수 없습니다**. 행을 유일하게 구분해야 하니까요.
- 참조 무결성(Referential Integrity)**: **외래 키 값**은 참조하는 테이블에 **실제로 존재하거나 NULL**이어야 합니다. 없는 값을 참조하면 위반.

암기 팁: 도메인(값의 범위), 개체(기본키 규칙), 참조(외래키 규칙). 어떤 키/값에 관한 규칙인지로 구분하세요.

3번

아래 내용에서 설명 글의 괄호 안의 용어를 영문 약자로 작성하시오.

()은/는 3글자의 영어 약자로 이루어진 오류 검출 코드로, 데이터에 체크섬을 추가하여 전송/저장 후 수신·읽기 시 다시 계산하여 변경 여부를 확인한다. 정해진 다항식($x^3 + x + 1$)을 기반으로 2진수 나눗셈을 하고 나머지를 () 값으로 삼는다.

정답: CRC

해설:

CRC(Cyclic Redundancy Check, 순환 중복 검사)는 데이터 전송 오류를 검출하는 대표적인 방법입니다.

- 송신 측에서 데이터를 **정해진 다항식으로 나눗셈(2진수 나눗셈)**한 뒤 **그 나머지를 검사값(CRC)**으로 만들어 데이터 뒤에 붙여 보냅니다.
- 수신 측에서 같은 방식으로 다시 계산해, 나머지가 0이 아니면 "전송 중 데이터가 변했다"고 판단합니다.
- 단순 패리티 검사보다 더 강력하게 오류를 잡아낼 수 있습니다.

키워드 "다항식 나눗셈 + 나머지 = 검사값 = CRC"입니다. 오류 검출(탐지)만 하고 정정은 하지 않는다는 점도 알아두세요. (오류 정정까지 하는 것은 Hamming 코드입니다.)

4번

다음은 악성코드 관련 문제이다. 보기에서 골라 작성하시오.

사용자가 원치 않는 소프트웨어를 구매하도록 사회공학을 사용하여 충격·불안·위협 인식을 유발하는 악성 소프트웨어. '겁을 주다'에서 유래했으며 가짜 바이러스 경고나 시스템 문제를 표시하여 돈을 지불하거나 특정 소프트웨어를 설치하도록 속인다.

보기: ㄱ. 컴포넌트 웨어 ㄴ. 유즈웨어 ㄷ. 셔블웨어 ㄹ. 스캐어 웨어 ㄴ. 안티 스파이 웨어 ㅂ. 네트웨어 ㅅ. 그룹웨어 ㅇ. 애드웨어

정답: ㄹ (스캐어웨어)

해설:
스캐어웨어(Scareware)는 'scare(겁주다)'에서 온 말로, **사용자를 겁먹게 만들어** 불필요한 소프트웨어를 사거나 설치하게 유도하는 악성 프로그램입니다.

- "당신의 PC가 바이러스에 감염되었습니다! 지금 클릭하세요!" 같은 **가짜 경고창**을 띄워 불안을 조성합니다.
- 이는 사람의 심리를 이용하는 **사회공학(Social Engineering)** 기법의 일종입니다.

비교:
- 애드웨어(Adware): 원치 않는 광고를 보여주는 프로그램.
- 스파이웨어(Spyware): 몰래 정보를 빼가는 프로그램.

키워드 "겁주기 + 가짜 경고 = 스캐어웨어"로 기억하세요.

5번

다음은 Java 코드에 대한 문제이다. 알맞는 출력값을 작성하시오.

```
public class Main {
    public static void main(String[] args) {
        int a=5, b=0;
        try {
            System.out.print(a/b);
        } catch(ArithmeticException e) {
            System.out.print("출력1");
        } catch(ArrayIndexOutOfBoundsException e) {
            System.out.print("출력2");
        } catch(NumberFormatException e) {
            System.out.print("출력3");
        } catch(Exception e) {
            System.out.print("출력4");
        } finally {
            System.out.print("출력5");
        }
    }
}
```

정답: 출력1출력5

해설:
자바의 예외 처리와 **0으로 나누기 예외**를 다루는 문제입니다.

- a/b 에서 b=0이므로 **0으로 나누기**가 일어납니다. 자바에서 정수를 0으로 나누면 `ArithmeticException` 이 발생합니다.
- catch 절을 위에서부터 확인 → **첫 번째 catch(ArithmeticException e) 가 일치** → "출력1" 출력.
- 한 번 잡으면 나머지 catch들은 모두 건너웁니다.
- `finally` 는 항상 실행 → "출력5" 출력.

따라서 "출력1출력5". `System.out.print(a/b)` 에서 나눗셈이 먼저 예외를 일으키므로 print는 아무것도 출력하지 못하고 곧장 catch로 넘어갑니다. 예외 종류(0으로 나누기 = `ArithmeticException`)를 알아두세요.

6번

아래 내용은 ARP/RARP에 대한 설명이다. 각 설명에 해당하는 것을 작성하시오.

(1)은/는 네트워크상에서 IP 주소를 MAC 주소로 변환하는 프로토콜이고, (2)은/는 MAC 주소를 IP 주소로 변환하는 프로토콜이다.

정답: (1) ARP / (2) RARP

해설:

ARP와 RARP는 IP 주소와 MAC 주소를 서로 변환하는 프로토콜입니다.

- IP 주소: 논리적 주소(인터넷에서 사용하는 주소).
- MAC 주소: 물리적 주소(네트워크 카드에 새겨진 고유 번호).
- **ARP(Address Resolution Protocol): IP 주소 → MAC 주소**로 변환. "이 IP를 가진 기기의 물리 주소가 뭐지?"를 알아냅니다.
- **RARP(Reverse ARP): 반대로 MAC 주소 → IP 주소**로 변환. 'Reverse(거꾸로)'라는 이름 그대로입니다.

암기 팁: "ARP는 IP를 MAC으로(보통 방향), RARP는 그 반대(Reverse)". 둘의 방향만 헷갈리지 않으면 됩니다.

7번

다음은 SQL 문제이다. 아래 두 테이블을 참고하여 쿼리 실행 결과를 작성하시오. (※ 원문에 테이블 이미지 포함)

```
SELECT name, incentive FROM emp, sal
WHERE emp.id = sal.id and incentives >= 500
```

정답:

name incentives
--- ---
이순신 1000

해설:

두 테이블을 **조인(JOIN)**하여 조건에 맞는 행을 뽑는 문제입니다.

- FROM emp, sal WHERE emp.id = sal.id → 이것은 두 테이블을 **공통 컬럼(id)으로 연결**하는 방식입니다. (선택표로 나열하고 WHERE로 연결 조건을 거는 옛 방식의 조인)
- and incentives >= 500 → 거기에 추가로 "인센티브가 500 이상인" 행만 남깁니다.

두 조건을 모두 만족하는 행을 찾으면, id가 일치하면서 인센티브가 500 이상인 사람은 "이순신(1000)"뿐입니다.

핵심: 조인은 두 테이블을 공통 키로 묶는 것이고, WHERE에 조인 조건과 일반 조건을 함께 쓸 수 있습니다.

8번

아래는 데이터베이스에 관련된 설명이다. 알맞는 용어를 보기에서 골라 작성하시오.

1. 릴레이션에서 속성의 개수를 의미 : (1)
2. 릴레이션에서 튜플의 개수를 의미 : (2)
3. 한 릴레이션의 속성이 다른 릴레이션의 기본 키를 참조할 때, 참조하는 속성 : (3)
4. 특정 속성에 대해 입력될 수 있는 값의 유형이나 범위를 의미하고 무결성을 보장하는 기준 : (4)

[보기] ㉠. domain ㉡. primary ㉢. degree ㉣. candidate ㉤. cardinality ㉥. attribute ㉦. foreign

정답: (1) ㉢ / (2) ㉤ / (3) ㉥ / (4) ㉠

해설:

데이터베이스 기본 용어를 영어로 묻는 문제입니다.

1. **degree(차수)**: 릴레이션의 **속성(열)의 개수**.
2. **cardinality(카디널리티)**: 릴레이션의 **튜플(행)의 개수**.
3. **foreign key(외래키)**: 다른 릴레이션의 **기본키를 참조**하는 속성.
4. **domain(도메인)**: 한 속성이 가질 수 있는 **값의 범위·유형**. 무결성의 기준이 됨.

자주 헷갈리는 짝:

- degree = 열 개수 ↔ cardinality = 행 개수
- domain = 값의 범위

- foreign(외래) = 참조, primary(기본) = 대표 키

영어 용어로도 출제되니 한글-영어를 함께 외워두면 좋습니다.

9번

IP 주소가 192.168.35.10, 서브넷 255.255.252.0인 PC에서 브로드캐스팅으로 다른 IP로 정보를 전달할 때 수신할 수 있는 IP를 보기에서 골라 모두 작성하시오.

[보기] ㉠. 192.168.34.1 ㉡. 192.168.32.19 ㉢. 192.168.35.200 ㉣. 192.168.33.138 ㉤. 192.168.35.50

정답: ㉠, ㉡, ㉢, ㉣, ㉤

해설:

브로드캐스트는 "같은 네트워크(서브넷) 안의 모든 기기"에게 한꺼번에 보내는 것입니다. 따라서 **같은 네트워크에 속한 IP들만** 수신할 수 있습니다. 핵심은 "어디까지가 같은 네트워크인가"를 서브넷 마스크로 계산하는 것입니다.

서브넷 마스크 255.255.252.0을 분석합니다.

- 세 번째 옥텟이 252 = 2진수 11111100 입니다. 즉 상위 6비트가 네트워크, 하위 2비트가 호스트용입니다.
- 한 네트워크가 차지하는 세 번째 옥텟의 범위는 **4씩 묶입니다** (256 - 252 = 4).
- 32, 33, 34, 35가 하나의 묶음(블록)입니다. (32부터 4개)

내 IP의 세 번째 옥텟은 35 → 속한 블록은 **세 번째 옥텟이 32~35인 네트워크**입니다. 즉 192.168.32.0 ~ 192.168.35.255가 같은 네트워크입니다.

보기를 확인하면:

- ㉠ 192.168.34.1, ㉡ 192.168.32.19, ㉢ 192.168.35.200, ㉣ 192.168.33.138, ㉤ 192.168.35.50

세 번째 옥텟이 모두 32~35 범위 안 → **전부 같은 네트워크** → 모두 수신 가능(㉠, ㉡, ㉢, ㉣, ㉤).

핵심: 서브넷 마스크로 네트워크 블록 크기를 구하고(여기선 4 단위), 같은 블록에 속하는지 확인합니다.

10번

다음은 C언어에 대한 문제이다. 알맞은 출력값을 작성하시오.

```
#include <stdio.h>
char Data[5] = {'B', 'A', 'D', 'E'};
char c;

int main(){
    int i, temp, temp2;
    c = 'C';
    printf("%d\n", Data[3]-Data[1]);

    for(i=0; i<5; ++i){
        if(Data[i] > c)
            break;
    }

    temp = Data[i];
    Data[i] = c;
    i++;

    for(; i<5; ++i){
        temp2 = Data[i];
        Data[i] = temp;
        temp = temp2;
    }

    for(i=0; i<5; i++){
        printf("%c", Data[i]);
    }
}
```

정답: 4 / BACDE

해설:
문자(char)는 내부적으로 아스키 코드 숫자로 저장된다는 점이 핵심입니다. (A=65, B=66, C=67, D=68, E=69, ...)

먼저 첫 출력: `Data[3] - Data[1] = 'E' - 'A' = 69 - 65 = 4.`

배열 초기 상태: `Data = {'B', 'A', 'D', 'E', '\0'}`(빈 자리는 0)

`c = 'C'`(67). 첫 for문은 **c보다 큰 첫 문자**를 찾습니다.
- `Data[0]='B'`(66) > `'C'`(67)? 아니오
- `Data[1]='A'`(65) > `'C'`? 아니오
- `Data[2]='D'`(68) > `'C'`? **예** → `i=2`에서 break

이제 `'C'`를 `i=2` 자리에 끼워 넣고, 원래 있던 값들을 한 칸씩 뒤로 밀니다(삽입 정렬과 비슷).
- `temp = Data[2]='D'` 보관, `Data[2]='C'`로 교체, `i`를 3으로.
- `i=3`: `temp2=Data[3]='E'` 보관, `Data[3]=temp('D')`, `temp='E'`.
- `i=4`: `temp2=Data[4]=빈값`, `Data[4]=temp('E')`, ...

결과 배열: B, A, C, D, E → 마지막 for문이 출력 → **"BACDE"**

종합 출력: `"4"`(줄바꿈) 다음 `"BACDE"`. `'C'`를 알맞은 위치에 끼워 넣어 정렬된 모양을 만드는 과정입니다.

11번

다음은 C언어에 대한 문제이다. 알맞은 출력값을 작성하시오.

```
#include <stdio.h>
#include <stdlib.h>

void set(int** arr, int* data, int rows, int cols) {
    for (int i = 0; i < rows * cols; ++i) {
        arr[((i + 1) / rows) % rows][((i + 1) % cols)] = data[i];
    }
}

int main() {
    int rows = 3, cols = 3, sum = 0;
    int data[] = {5, 2, 7, 4, 1, 8, 3, 6, 9};
    int** arr;
    arr = (int**) malloc(sizeof(int*) * rows);

    for (int i = 0; i < cols; i++) {
        arr[i] = (int*) malloc(sizeof(int) * cols);
    }

    set(arr, data, rows, cols);

    for (int i = 0; i < rows * cols; i++) {
        sum += arr[i / rows][i % cols] * (i % 2 == 0 ? 1 : -1);
    }

    for(int i=0; i<rows; i++) {
        free(arr[i]);
    }
    free(arr);

    printf("%d", sum);
}
```

정답: 13

해설:
동적 2차원 배열에 데이터를 특정 위치에 배치한 뒤, 짝수 인덱스는 더하고 홀수 인덱스는 빼는 문제입니다.

먼저 set 함수가 data를 arr의 각 칸에 어떤 위치로 넣는지 계산합니다(`rows=cols=3`). 각 `i`에 대해 `행 = ((i+1)/3)%3`, `열 = (i+1)%3` :
- `i=0`: `arr[0][1]=5`

```
- i=1: arr[0][2]=2
- i=2: arr[1][0]=7
- i=3: arr[1][1]=4
- i=4: arr[1][2]=1
- i=5: arr[2][0]=8
- i=6: arr[2][1]=3
- i=7: arr[2][2]=6
- i=8: arr[0][0]=9
```

배열 완성:

```
arr[0] = {9, 5, 2}
arr[1] = {7, 4, 1}
arr[2] = {8, 3, 6}
```

두 번째 루프는 `arr[i/3][i%3]` 을 순서대로 읽으며(9,5,2,7,4,1,8,3,6), 짝수 인덱스(+) 홀수 인덱스(-)로 합산:

- +9 -5 +2 -7 +4 -1 +8 -3 +6

- = (9+2+4+8+6) - (5+7+1+3) = 29 - 16 = **13**

핵심: 인덱스 계산을 차근차근 표로 정리하고, 부호(짝수 +, 홀수 -)를 번갈아 적용하면 됩니다.

12번

다음은 결합도와 관련된 내용이다. 보기에서 알맞은 답을 골라 작성하시오.

- (1) 다른 모듈 내부에 있는 변수나 기능을 다른 모듈에서 사용하는 경우의 결합도
- (2) 모듈 간의 인터페이스로 배열이나 오브젝트, 자료구조 등이 전달되는 경우의 결합도
- (3) 파라미터가 아닌 모듈 밖에 선언된 전역 변수를 참조·갱신하는 식으로 상호작용하는 경우의 결합도

[보기] ㄱ. 자료 결합도 ㄴ. 스탬프 결합도 ㄷ. 제어 결합도 ㄹ. 공통 결합도 ㄴ. 내용 결합도 ㅂ. 외부 결합도

정답: (1) ㄴ / (2) ㄴ / (3) ㄹ

해설:

결합도(Coupling)는 모듈 간 의존성으로, **낮을수록 좋은** 설계입니다. 종류별 정의를 구분합시다.

- 1. **내용 결합도(Content)**: 한 모듈이 다른 모듈의 **내부(변수·기능)에 직접 접근**. 결합도가 **가장 높음(최악)**.
- 2. **스탬프 결합도(Stamp)**: **배열·구조체 같은 자료구조 전체**를 인터페이스로 전달. (그중 일부만 쓰더라도 전체를 넘김)
- 3. **공통 결합도(Common)**: **전역 변수**를 여러 모듈이 함께 참조·수정.

결합도 강한 순서(나쁨→ 좋음):

내용 > 공통 > 외부 > 제어 > 스탬프 > 자료

- 구분 팁:
- 내용: 남의 내부를 직접 건드림(최악)
 - 스탬프: 자료구조 통째로 전달
 - 공통/외부: 전역 변수 공유
 - 자료: 순수 데이터만 전달(최선)

13번

다음은 Java 코드에 대한 문제이다. 알맞는 출력값을 작성하시오.

```
public class Main {
    public static void main(String[] args) {
        new Child();
        System.out.println(Parent.total);
    }
}

class Parent {
    static int total = 0;
```

```

    int v = 1;
    public Parent() {
        total += (++v);
        show();
    }
    public void show() {
        total += total;
    }
}

class Child extends Parent {
    int v = 10;
    public Child() {
        v += 2;
        total += v++;
        show();
    }
    @Override
    public void show() {
        total += total * 2;
    }
}

```

정답: 54

해설:
생성자 호출 순서, 오버라이딩, static 변수 공유가 모두 얹힌 고난도 문제입니다. 차근차근 추적합니다. total은 static이라 하나만 존재하며 공유됩니다.

`new Child()` → 자식 생성자는 먼저 부모 생성자를 호출(`super()` 자동)합니다.

- 【1단계】Parent() 생성자 실행:
- `total += (++v)` → Parent의 v는 1, ++v로 2가 되고 `total = 0 + 2 = 2`.
 - `show()` 호출 → **여기가 함정!** 실제 객체는 Child이므로 **Child의 show()**가 호출됩니다(**오버라이딩, 동적 바인딩**).
 - Child의 `show()`: `total += total * 2` → `total = 2 + (22) = 2 + 4 = 6`.

- 【2단계】Child() 생성자 실행:
- `v += 2` → Child의 v는 10, +2로 12. (Parent의 v와 별개)
 - `total += v++` → `total = 6 + 12 = 18`. (v는 후위증가라 18 더한 뒤 13이 됨)
 - `show()` → Child의 `show()`: `total = 18 + (182) = 18 + 36 = 54`.

최종 total = **54**.

핵심 포인트: 부모 생성자 안의 `show()` 도 **실제 객체(Child)의 오버라이드된 show()**를 부른다는 점입니다. 이것이 가장 큰 함정입니다.

14번

아래는 디자인 패턴에 대한 설명이다. 알맞는 답을 보기에서 골라 작성하시오. (※ 원문에 이미지 포함)

서로 다른 인터페이스를 가진 클래스들을 연결해 사용 가능하게 한다. 기존 클래스(Adaptee)를 원하는 인터페이스(Target)에 맞게 변환하는 어댑터(Adapter)를 만들며, 기존 클래스를 감싸서(wrapper) 인터페이스를 변환한다.

정답: Adapter

해설:
Adapter(어댑터) 패턴은 **호환되지 않는 인터페이스를 가진 클래스들을 연결**해주는 구조 패턴입니다.

- 현실의 '돼지코' 어댑터(110V↔220V 변환)를 떠올리면 됩니다. 모양이 안 맞는 것을 중간에서 맞춰주는 역할입니다.
- 기존 클래스(Adaptee)를 직접 수정하지 않고, 그것을 **감싸는(wrapper) 어댑터**를 만들어 원하는 인터페이스(Target)에 맞게 변환합니다.
- 덕분에 기존 코드를 건드리지 않고 새 인터페이스와 함께 쓸 수 있습니다.

키워드 "서로 다른 인터페이스 연결 + wrapper + 변환 = Adapter"입니다. 구조(Structural) 패턴에 속합니다.

15번

문장(Statement) 커버리지 테스트를 수행하려고 한다. 코드를 아래의 제어 흐름도 빈칸에 연결되도록 작성하고 문장 커버리지 순서대로 작성하시오. (※ 원문에 흐름도 이미지 포함)

```
int Main(int b[], int m, int x) {
    int a = 0;
    while (a < m || b[a] < x) {
        if (b[a] < 0)
            b[a] = -b[a];
        a++;
    }
    return 1;
}
```

1. (①) 2. (②) 3. (③) 4. (④) 5. (⑤) 6. (⑥)
문장 커버리지 순서 1 → 2 → (⑦)

정답:
- (1) int a = 0
- (2) a < m || b[a] < x
- (3) b[a] < 0
- (4) b[a] = -b[a];
- (5) a++;
- (6) return 1;
- (7) ③ → ④ → ⑤ → ② → ⑥

해설:
제어 흐름 그래프(Control Flow Graph)는 프로그램의 실행 흐름을 노드(문장)와 화살표(흐름)로 그린 것입니다. 각 코드 줄을 노드 번호에 매핑하는 문제입니다.

- 코드를 흐름 순서대로 노드에 배치하면:
- ① int a = 0 : 시작 초기화
 - ② a < m || b[a] < x : while의 조건 판단(분기점)
 - ③ b[a] < 0 : if의 조건 판단
 - ④ b[a] = -b[a]; : if가 참일 때 실행(절댓값으로 변환)
 - ⑤ a++; : 반복 변수 증가
 - ⑥ return 1; : while을 빠져나간 뒤 종료

문장 커버리지는 모든 문장을 한 번씩만 실행하면 되는 가장 약한 기준입니다. 시작(①→②) 후 반복문 내부를 한 바퀴 돌며 모든 문장을 거치는 순서는:
③(if 조건) → ④(if 실행) → ⑤(증가) → ②(조건 재확인) → ⑥(종료)

핵심: 코드의 논리적 흐름(초기화 → 조건 → 분기 → 본문 → 증가 → 복귀 → 종료)을 그래프 노드에 대응시키는 연습입니다.

16번

다음은 Java 코드에 대한 문제이다. 알맞는 출력값을 작성하시오.

```
public class Main {
    public static void main(String[] args) {
        int[] data = {3, 5, 8, 12, 17};
        System.out.println(func(data, 0, data.length - 1));
    }

    static int func(int[] a, int st, int end) {
        if (st >= end) return 0;
        int mid = (st + end) / 2;
        return a[mid] + Math.max(func(a, st, mid), func(a, mid + 1, end));
    }
}
```

정답: 20

해설:
분할 정복(재귀) 문제입니다. 배열을 반으로 나눠 가며 중간값을 더하고, 양쪽 중 더 큰 결과를 선택합니다.

배열: a = {3, 5, 8, 12, 17} (인덱스 0~4)

- 호출을 추적합니다. func(0, 4) :
- mid = (0+4)/2 = 2 → a[2] = 8
 - 왼쪽 func(0, 2) 계산:
 - mid = 1 → a[1] = 5
 - func(0,1): mid=0 → a[0]=3, func(0,0)→0, func(1,1)→0 → 3+max(0,0)=3
 - func(2,2): st>=end → 0
 - = 5 + max(3, 0) = 8
 - 오른쪽 func(3, 4) 계산:
 - mid = 3 → a[3] = 12
 - func(3,3)→0, func(4,4)→0 → 12 + max(0,0) = 12
 - 전체 = a₂ + max(왼쪽 8, 오른쪽 12) = 8 + 12 = **20**

핵심: 재귀는 가장 깊은 곳(st >= end)부터 0을 반환하며 거슬러 올라옵니다. Math.max 로 두 갈래 중 큰 값을 누적합니다.

17번

다음은 파이썬에 대한 문제이다. 알맞는 출력값을 작성하시오.

```
class Node:
    def __init__(self, value):
        self.value = value
        self.children = []

def tree(li):
    nodes = [Node(i) for i in li]
    for i in range(1, len(li)):
        nodes[(i - 1) // 2].children.append(nodes[i])
    return nodes[0]

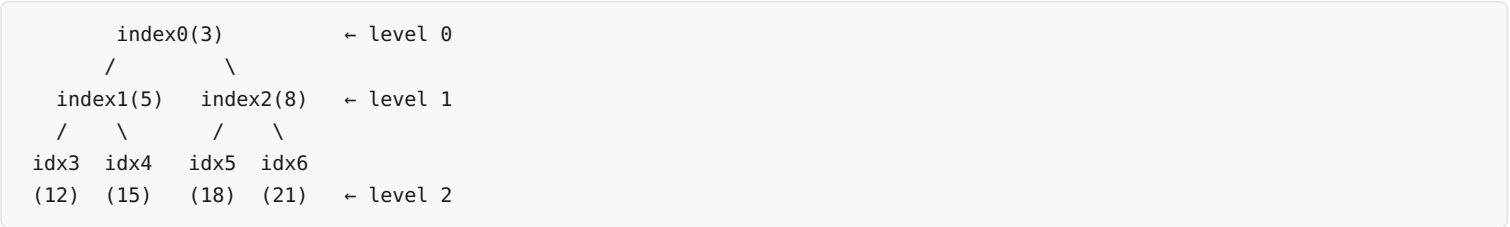
def calc(node, level=0):
    if node is None:
        return 0
    return (node.value if level % 2 == 1 else 0) + sum(calc(n, level + 1) for n in node.children)

li = [3, 5, 8, 12, 15, 18, 21]
root = tree(li)
print(calc(root))
```

정답: 13

해설:
배열을 이진 트리로 만든 뒤, 홀수 레벨에 있는 노드의 값만 더하는 문제입니다.

tree 함수는 배열을 완전 이진 트리로 만듭니다. (i-1)//2 가 부모 인덱스이므로:



- calc 는 level % 2 == 1 (홀수 레벨)일 때만 값을 더합니다.
- level 0 (값 3): 짝수 레벨 → 더하지 않음 (0)
 - level 1 (값 5, 8): 홀수 레벨 → **5 + 8 = 13** 더함
 - level 2 (값 12, 15, 18, 21): 짝수 레벨 → 더하지 않음

따라서 합 = **13**.

핵심: 배열 인덱스로 부모/자식 관계를 만드는 공식(부모 = (i-1)//2)과, 레벨의 짝/홀 판별(level % 2)을 이해하면 됩니다.

18번

다음은 C언어에 대한 문제이다. 알맞는 출력값을 작성하시오.

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Data {
    int value;
    struct Data *next;
} Data;

Data* insert(Data* head, int value) {
    Data* new_node = (Data*)malloc(sizeof(Data));
    new_node->value = value;
    new_node->next = head;
    return new_node;
}

Data* reconnect(Data* head, int value) {
    if (head == NULL || head->value == value) return head;
    Data *prev = NULL, *curr = head;
    while (curr != NULL && curr->value != value) {
        prev = curr;
        curr = curr->next;
    }
    if (curr != NULL && prev != NULL) {
        prev->next = curr->next;
        curr->next = head;
        head = curr;
    }
    return head;
}

int main() {
    Data *head = NULL, *curr;
    for (int i = 1; i <= 5; i++)
        head = insert(head, i);
    head = reconnect(head, 3);
    for (curr = head; curr != NULL; curr = curr->next)
        printf("%d", curr->value);
    return 0;
}
```

정답: 35421

해설:

연결 리스트에서 **특정 값을 가진 노드를 맨 앞으로 옮기는** 문제입니다.

먼저 insert를 5번 반복: `new_node->next = head` 로 **새 노드를 맨 앞에 붙이는** 방식(앞쪽 삽입)입니다.

- 1 삽입 → [1]
- 2 삽입 → [2,1]
- 3 삽입 → [3,2,1]
- 4 삽입 → [4,3,2,1]
- 5 삽입 → **[5,4,3,2,1]**

이제 `reconnect(head, 3)` : 값이 3인 노드를 찾아 **맨 앞으로 이동**시킵니다.

- 3을 찾을 때까지 이동: `prev=4` 노드, `curr=3` 노드.
- `prev->next = curr->next` → 4의 다음을 3의 다음(=2)으로 연결 → 3을 리스트에서 떼어냄. 리스트: [5,4,2,1]
- `curr->next = head; head = curr;` → 3을 맨 앞에 붙임 → **[3,5,4,2,1]**

마지막 출력 루프가 순서대로 출력 → **"35421"**

핵심: 노드를 옮길 때 ① 떼어내기(앞 노드를 뒤 노드에 연결), ② 맨 앞에 붙이기(head 갱신)의 두 단계를 거칩니다.

19번

다음은 C언어에 대한 문제이다. 알맞는 출력값을 작성하시오.

```
#include <stdio.h>

typedef struct student {
    char* name;
    int score[3];
} Student;

int dec(int enc) {
    return enc & 0xA5;
}

int sum(Student* p) {
    return dec(p->score[0]) + dec(p->score[1]) + dec(p->score[2]);
}

int main() {
    Student s[2] = { "Kim", {0xA0, 0xA5, 0xDB}, "Lee", {0xA0, 0xED, 0x81} };
    Student* p = s;
    int result = 0;
    for (int i = 0; i < 2; i++) {
        result += sum(&s[i]);
    }
    printf("%d", result);
    return 0;
}
```

정답: 908

해설:
비트 AND 연산(&)을 16진수에 적용하는 문제입니다. 각 점수를 0xA5와 AND 연산한 뒤 모두 더합니다.

0xA5를 2진수로 보면 1010 0101 입니다. AND 연산은 두 비트가 모두 1일 때만 1입니다.

- 각 값을 0xA5(=10100101)와 AND 합니다:
- 0xA0(10100000) & 0xA5 = 10100000 = 0xA0 = **160**
 - 0xA5(10100101) & 0xA5 = 10100101 = 0xA5 = **165**
 - 0xDB(11011011) & 0xA5 = 10000001 = 0x81 = **129**
 - 0xA0(10100000) & 0xA5 = 10100000 = 0xA0 = **160**
 - 0xED(11101101) & 0xA5 = 10100101 = 0xA5 = **165**
 - 0x81(10000001) & 0xA5 = 10000001 = 0x81 = **129**

합계 = 160+165+129+160+165+129 = **908**

핵심: 16진수를 2진수로 풀어 0xA5와 자리별 AND를 수행하고, 결과를 10진수로 바꿔 더합니다. 비트 연산은 자리(비트)별로 따로 계산한다는 점이 중요합니다.

20번

다음은 Java 코드에 대한 문제이다. 알맞는 출력값을 작성하시오.

```
public class Main {
    public static void main(String[] args) {
        System.out.println(calc("5"));
    }

    static int calc(int value) {
        if (value <= 1) return value;
        return calc(value - 1) + calc(value - 2);
    }

    static int calc(String str) {
```

```
int value = Integer.valueOf(str);
if (value <= 1) return value;
return calc(value - 1) + calc(value - 3);
}
}
```

정답: 4

해설:

오버로딩된 두 개의 **calc**가 있습니다. 처음 호출은 문자열 "5"이므로 `calc(String)` 이 불립니다. 하지만 그 안에서 `calc(value-1)` 처럼 정수를 넘기면 `calc(int)` 가 호출됩니다. 즉 한 번 int 버전으로 들어가면 계속 int 버전만 씁니다.

먼저 `calc("5")` → `value=5` → `calc(4) + calc(2)` (int 버전 호출).

이제 `calc(int)` 는 피보나치와 비슷한 재귀입니다: `calc(n) = calc(n-1) + calc(n-2)` , $n \leq 1$ 이면 n 반환.

- `calc(0)=0`, `calc(1)=1`
- `calc(2)=calc(1)+calc(0)=1+0=1`
- `calc(3)=calc(2)+calc(1)=1+1=2`
- `calc(4)=calc(3)+calc(2)=2+1=3`

따라서 `calc("5") = calc(4) + calc(2) = 3 + 1 = 4`.

핵심: 첫 호출만 String 버전이고, 그 내부의 재귀는 모두 int 버전(`calc(n-1)+calc(n-2)`)을 탑니다. String 버전의 `value-3` 공식은 처음 한 번만 적용되어 `calc(4)+calc(2)`로 분기된다는 점을 정확히 봐야 합니다.